

Web Application Security Assessment Report

SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

Prepared by Mohammed Ashraf Mokhtar

Prepared By	Mohammed Ashraf Mokhtar
LinkedIn	linkedin.com/in/1mohammed-ashraf
GitHub	github.com/moashraf2006
Target Application	we-like-to-shop.net (PortSwigger Web Security Academy training environment)
Host	0a9b001d04699e2d80f830e9005000e8.web-security-academy.net
Assessment Type	Manual black-box penetration test — SQL Injection
Vulnerable Endpoint	GET /filter?category=
Tooling	Burp Suite Community Edition v2026.7.3 (Proxy, Repeater)
Classification	CWE-89: Improper Neutralization of Special Elements used in an SQL Command
Severity	High
Status	Confirmed — Exploited Successfully — Lab Solved
Date of Testing	2 September 2026

This document is a technical write-up of a controlled exercise performed against a PortSwigger Web Security Academy training lab. All testing was conducted against an intentionally vulnerable, isolated instance provided for educational purposes. No production systems or real user data were involved.

1. Executive Summary

During testing of the target application's product catalogue filtering feature, a SQL injection vulnerability was identified in the **category** parameter of the `/filter` endpoint. The application concatenates user-supplied input directly into a server-side SQL query without sanitisation or parameterisation, allowing an attacker to alter the logic of the underlying WHERE clause.

By submitting a crafted payload that always evaluates to true, the filter condition was bypassed entirely, causing the application to return **all** products in the catalogue — including unreleased items that are not intended to be visible to end users. This confirms that an unauthenticated attacker can read data they are not authorised to access, satisfying the lab's success condition.

Impact: Unauthorised disclosure of hidden/unreleased application data via unauthenticated SQL injection. Depending on database privileges, this class of vulnerability can typically be extended to full database enumeration, authentication bypass, and in some configurations, remote code execution.

HIGH

Key Findings

- The category GET parameter is inserted into a SQL query without input validation or use of parameterised queries.
- A single quote character breaks out of the intended string literal, enabling injection of arbitrary SQL logic.
- The payload ' **OR 1=1** - - forces the WHERE clause to evaluate to true for every row, bypassing the category and release-status filters.
- The vulnerability was confirmed and successfully exploited using Burp Suite Repeater; the lab was marked **Solved**.

2. Scope & Methodology

2.1 Scope

Testing was limited to the single-page 'We Like To Shop' e-commerce demo application hosted on the PortSwigger Web Security Academy platform, specifically the product category filtering functionality reachable at `/filter?category=<value>`.

2.2 Methodology

Traffic between the browser and the target was intercepted and analysed using Burp Suite's Proxy tool. Interesting requests were sent to Repeater for iterative, manual manipulation of the category parameter to probe for SQL injection. The standard black-box SQL injection workflow was followed:

- 1 Map the application's parameters and identify a data-driven feature (category filter) likely to interact with a backend database.
- 2 Intercept a baseline request in Burp Proxy and forward it to Repeater to establish normal, expected behaviour.
- 3 Submit boundary-testing characters (a single quote) to check for a change in application behaviour or error response, confirming the input reaches a SQL query unsanitised.
- 4 Iterate on the payload to construct a syntactically valid injection that alters the query logic (a tautology / always-true condition).
- 5 Send the crafted request and verify, via the rendered response, that the filter was bypassed and hidden data was returned.
- 6 Confirm exploitation success against the lab's defined objective.

2.3 Tools Used

Interception Proxy	Burp Suite Community Edition v2026.7.3
Modules	Proxy (Intercept / HTTP history), Repeater
Browser	Chromium-based browser (Burp-embedded), Windows 10 / Edge UA

3. Vulnerability Details

Vulnerability	SQL Injection (in-band, error-less / boolean tautology)
CWE Reference	CWE-89 — Improper Neutralization of Special Elements used in an SQL Command
OWASP Category	A03:2021 — Injection
Affected Parameter	category (GET, /filter endpoint)
Authentication Required	None — publicly reachable, unauthenticated
CVSS 3.1 (indicative)	7.5 (High) — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

3.1 Root Cause

When a user selects a category, the application builds and executes a server-side SQL statement by directly concatenating the unsanitised category parameter into the query string, in the following form:

SQL — server-side query (reconstructed)

```
SELECT * FROM products
WHERE category = 'Gifts' AND released = 1
```

Because the parameter value is placed inside the string literal without escaping or parameter binding, any single quote supplied by the client terminates the literal early, and any text following it is interpreted by the database as SQL syntax rather than as data.

3.2 Proof of Concept — Evidence Log

Step 1 — Lab reconnaissance. The lab brief confirms the target query structure and success condition: cause the application to disclose one or more unreleased products via the category filter.

The screenshot shows the PortSwigger Web Security Academy interface. On the left, a 'My progress' sidebar lists several steps, with 'Retrieving hidden data (3 of 3)' currently selected. The main content area is titled 'Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data'. It includes a 'Solved' badge, a description of the lab's goal, and the target SQL query: `SELECT * FROM products WHERE category = 'Gifts' AND released = 1`. Below the query, there is an 'ACCESS THE LAB' button and a 'Solution' dropdown menu. At the bottom, there are 'BACK' and 'CONTINUE' buttons, with a note indicating the next step is 'Subverting application logic'.

Figure 1. PortSwigger lab brief for “SQL injection vulnerability in WHERE clause allowing retrieval of hidden data,” showing the target query and success condition.

Step 2 — Baseline behaviour. Browsing the target's “We Like To Shop” storefront confirms the lab is not yet solved, and that a Refine your search panel drives the category parameter.

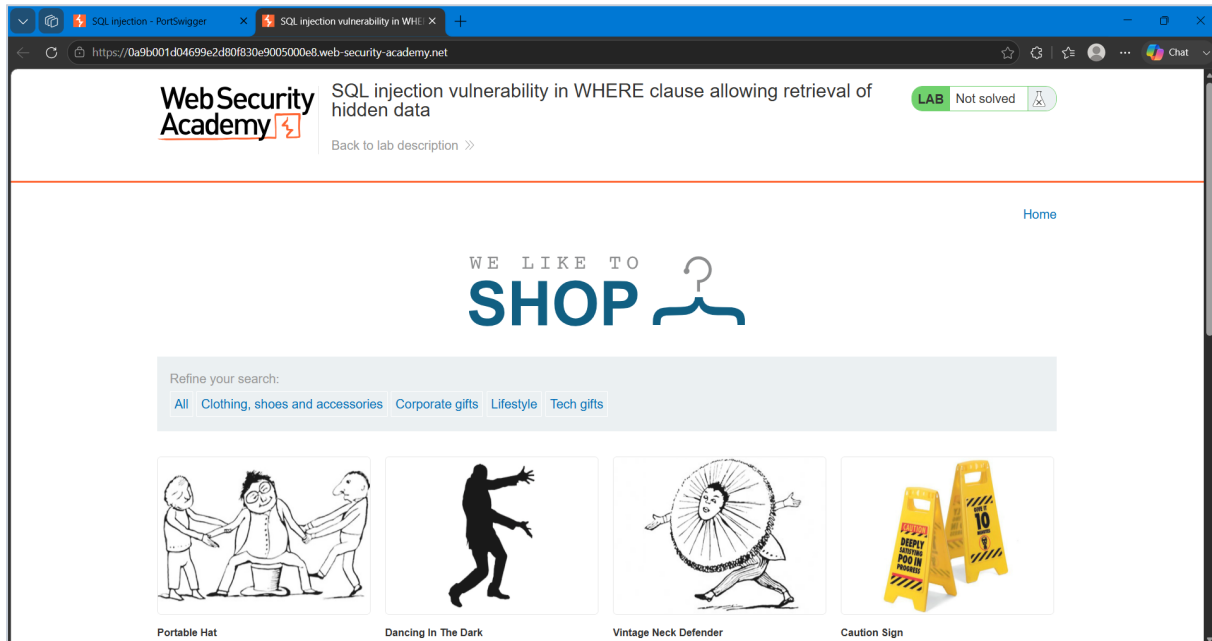


Figure 2. Target application in its default state — LAB status “Not solved.”

Selecting the **Lifestyle** category confirms the parameter is reflected in the URL as `/filter?category=Lifestyle`, giving a concrete injection point to test.

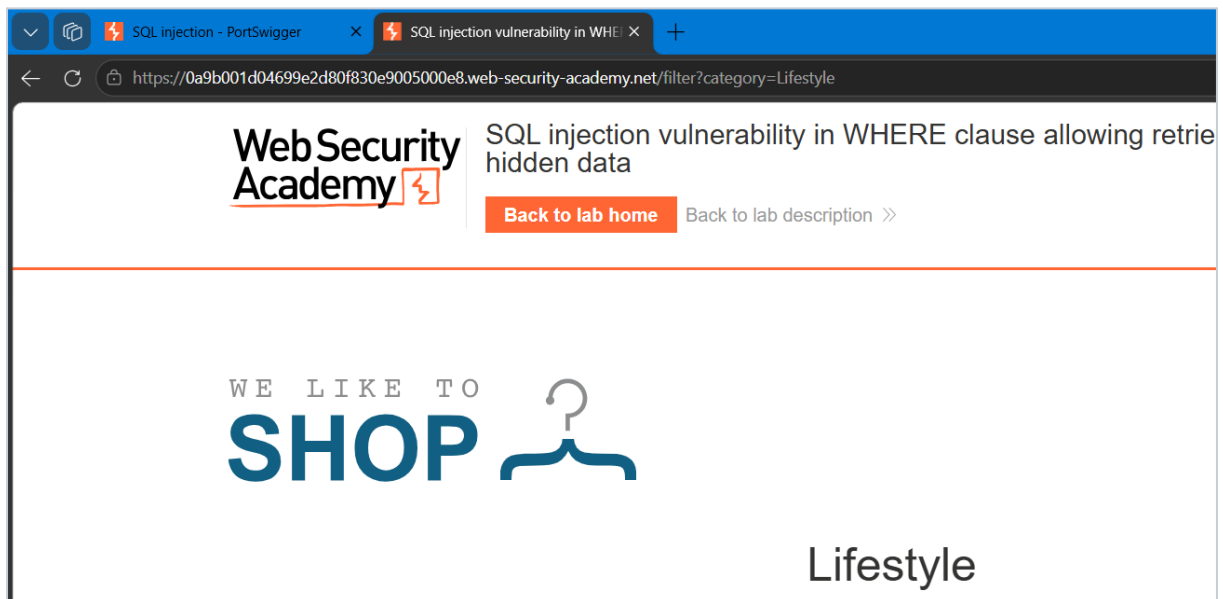


Figure 3. Baseline request generated by selecting the “Lifestyle” category filter.

Step 3 — Intercept the request. With Burp Proxy's intercept enabled, the `GET /filter?category=Lifestyle` request is captured before it reaches the server, ready to be sent to Repeater for manipulation.

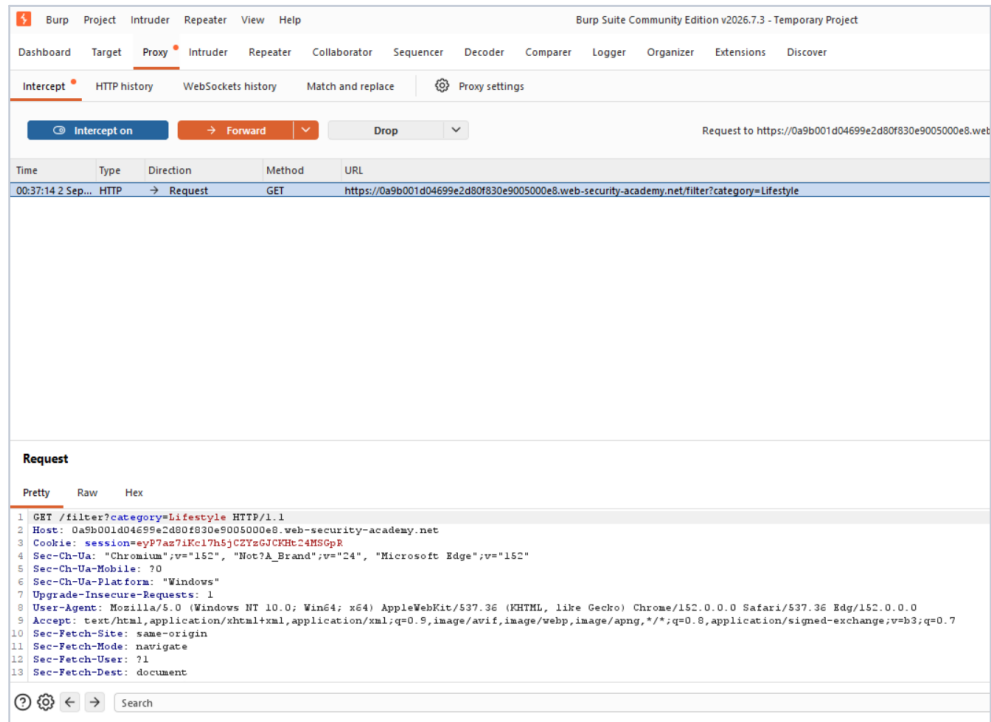


Figure 4. Burp Suite Proxy → Intercept, showing the captured baseline request (GET /filter?category=Lifestyle).

Step 4 — Baseline in Repeater. The captured request is sent to Repeater, preserving all original headers and the session cookie, to allow controlled, repeatable modification of the category parameter.

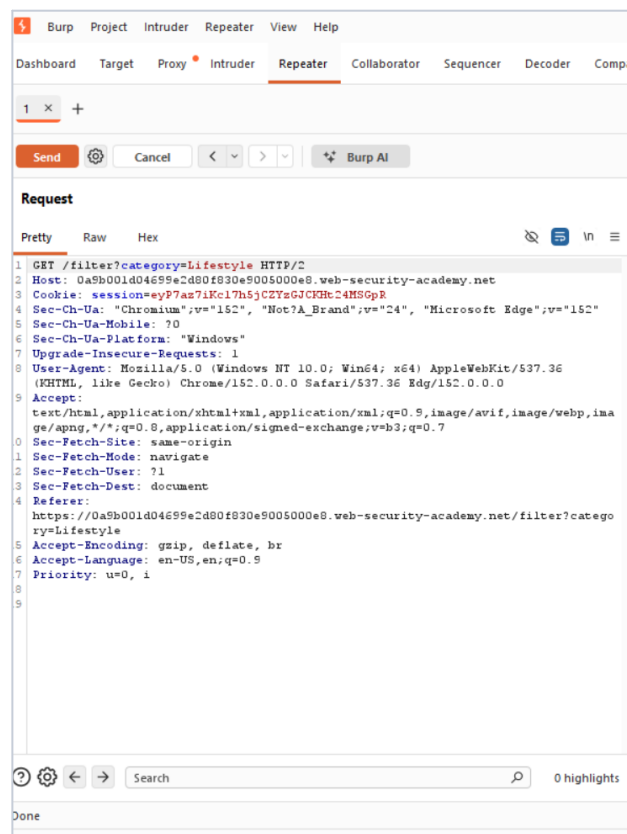


Figure 5. Burp Suite Repeater loaded with the baseline request prior to payload injection.

Step 5 — Injection payload and result. The category value was replaced with a payload designed to close the string literal and append a condition that always evaluates to true, commenting out the remainder of the original query:

Payload used

```
category=' OR 1=1--
```

Resulting query interpreted by the database:

SQL — effective query after injection

```
SELECT * FROM products
WHERE category = ' OR 1=1-- ' AND released = 1
```

The trailing double-dash (--) comments out the rest of the original statement (including the AND released = 1 clause), and the tautology 1=1 makes the WHERE clause true for every row in the table.

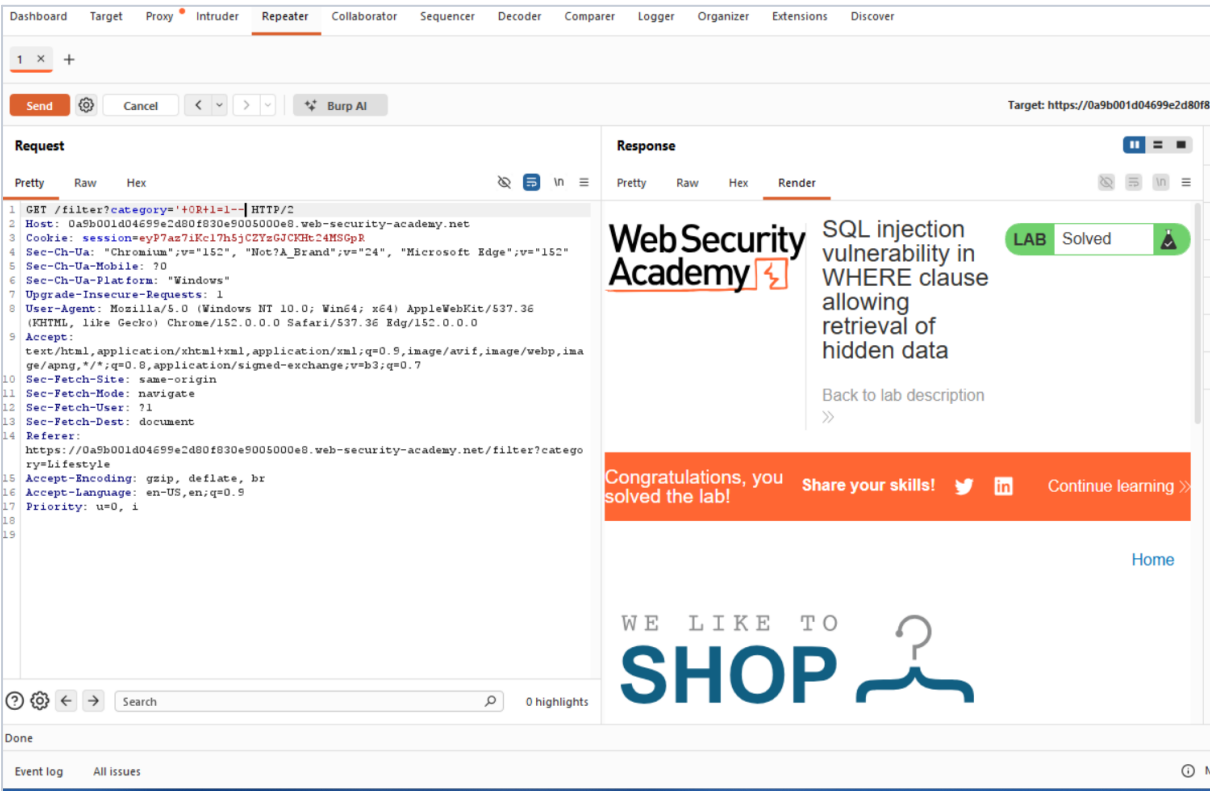


Figure 6. Burp Suite Repeater — malicious request (GET /filter?category=' OR 1=1--) sent on the left; rendered response on the right shows the full, unfiltered catalogue and the LAB status flips to “Solved,” with the application’s own “Congratulations, you solved the lab!” banner confirming successful exploitation.

Result Summary

HTTP Response	200 OK
Observed Effect	Category and release-status filters bypassed; all catalogue items (including unreleased products) rendered on the page
Lab Status	Solved — success banner returned: “Congratulations, you solved the lab!”

4. Impact Assessment

Although this instance discloses only unreleased product listings, the underlying flaw — unsanitised concatenation of user input into a SQL statement — is a critical class of vulnerability. In a real-world deployment with a similarly structured query, the same technique could realistically be extended to:

- Enumerate the database schema and extract arbitrary tables (e.g. via UNION-based injection).
 - Read sensitive data such as user credentials, session tokens, or personal information.
 - Bypass authentication logic if the same pattern is present in a login query.
 - Depending on database engine and permissions, modify or delete data, or escalate to command execution.
- Because the endpoint requires no authentication, the vulnerability is exploitable by any anonymous visitor, which further increases its severity.

5. Remediation

The following corrective actions are recommended, in priority order:

- 1 **Use parameterised queries / prepared statements** for all database access, so that user input is always treated as data and never as executable SQL syntax. Example (illustrative): `SELECT * FROM products WHERE category = ? AND released = 1`, with the category value bound as a parameter.
- 2 **Adopt an ORM or query-builder** that parameterises input by default, reducing the risk of ad-hoc string concatenation reappearing elsewhere in the codebase.
- 3 **Apply strict server-side input validation** (e.g. allow-listing known category values) as a defence-in-depth measure, in addition to parameterisation — never as a substitute for it.
- 4 **Apply the principle of least privilege** to the database account used by the application, limiting its ability to read tables or columns beyond what the feature requires.
- 5 **Deploy a Web Application Firewall (WAF)** as a compensating control to detect and block common SQL injection patterns while remediation is rolled out.
- 6 **Conduct a codebase-wide review** for the same anti-pattern, since a single instance of unsanitised query construction often indicates the pattern exists elsewhere in the application.

6. Conclusion

This assessment confirmed a High-severity SQL injection vulnerability in the product category filter of the target application. The vulnerability was successfully exploited using a boolean-based tautology payload, resulting in disclosure of data that should not be accessible to unauthenticated users. The lab objective was met and the exercise was marked as **Solved** by the platform.

Remediation should prioritise migrating all dynamic SQL construction to parameterised queries, which eliminates this class of vulnerability at the root cause rather than relying solely on input filtering.

Reference

PortSwigger Web Security Academy — “SQL injection vulnerability in WHERE clause allowing retrieval of hidden data”, Retrieving hidden data (3 of 3), SQL injection module, lab 6 of 51.